CSCI3170
Introduction to Database Systems

7-functional_dependencies

Slides: Michael Yu

Motivating Example

- Let's say the values of hourly_wages depend directly on that of rating.
  - Rating = 8 will all receive hourly_wages = 10
  - Rating = 5 will all receive hourly_wages = 7

- Expect Redundant Storage
  - The rating value 8 corresponds to the hourly wage 10, and this association is repeated three times.
  - Storage is not used efficiently.

Id	name	age	rating	Hourly_wages	Hours_worked
123-22-3666	Peter	48	8	10	40
231-31-5368	Paul	22	8	10	30
131-24-3650	Mary	35	5	7	30
434-26-3751	David	35	5	7	32
612-67-4134	Ada	35	8	10	40

M: is this schema technically incorrect?

Other Cases for Issue?

- Expect Insertion Anomalies
  - E.g., a new staff inserting a tuple for an employee without knowing the hourly wage for the employee's rating value.
  - *null* values may appear in the table.

- Expect Deletion Anomalies
  - If we delete all tuples with a given rating value (e.g. tuples of Mary and David) we lose the association between the rating value and its hourly_wage value.

Id	name	age	rating	Hourly_wages	Hours_worked
123-22-3666	Peter	48	8	10	40
231-31-5368	Paul	22	8	10	30
131-24-3650	Mary	35	5	7	30
434-26-3751	David	35	5	7	32
612-67-4134	Ada	35	8	10	40

Latent Problems

1. Some information is stored repeatedly
   (i.e., Redundant Storage)

2. It may not be possible to store certain information correctly unless some other, supplemental, information is stored
   (i.e., Insertion anomalies)

3. It may not be possible to delete certain information without losing some other, unrelated, information
   (i.e., Deletion Anomalies)

4. If one copy of such repeated data is updated, an inconsistency is created, unless all copies are similarly updated
   (i.e., Update Anomalies)

Where is the issue Exactly?

- Is this a problem stemming from the data (or the from the schema we designed)?

Enrollment				
id	Course_title	course_code	Offering_dept	prof
1	Database	353	Comp Sci	Smith
2	Chem101	427	Chemistry	Turner
3	Database	353	Comp Sci	Clark

- Problematic update:

Enrollment				
id	Course_title	course_code	Offering_dept	prof
1	Database	353	Comp Sci	Smith
2	Chem101	427	Chemistry	Turner
3	DataScience	353	Comp Sci	Clark

How know if susceptible to issues?

- It unfortunately will require a keen understanding beyond the schema; a understanding of the data when the database is deployed live.
  - It is hard to know when you are a user of one other's dbms.
  - If the dbms was yours, we expect you to avoid these mistakes as early as the design stage.

ID	*name*	*salary*	*dept_name*	*building*	*budget*
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
22222	Einstein	95000	Biology	Watson	90000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

Hinting at the nature of Problem

- **Main Observation:** Issues arise naturally when there is redundancy. Redundancy arise when a relational schema forces a present association between attributes.

- **Goal:** Continuing from schema we built from E/R modelling, we are going to expand more on:
  - How to decide when a relation R has "good" form
  - Functional dependencies can start to help us identify such situations and suggest refinements to the schema.

Id	name	age	rating	Hourly_wages	Hours_worked
123-22-3666	Peter	48	8	10	40
231-31-5368	Paul	22	8	10	30
131-24-3650	Mary	35	5	7	30
434-26-3751	David	35	5	7	32
612-67-4134	Ada	35	8	10	40

Functional Dependencies

- Any functional dependency hints a **relation** between attributes
  - FD constraint: Whenever any two tuples t_1 and t_2 of r agree on one attribute α, they also agree on another attribute β.
    - $t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$
  - This is also denoted as $\alpha \rightarrow \beta$

- **Example**: R = {ID, Name, Code, Grade}

ID	Name	Code	Grade
100	J	3550	A
200	X	3550	B
100	J	4540	B
100	J	4550	A

How to incorporate them into the schema? Enforce?

- ID → Name
- ID → Grade
- ID → Code
- ID, Code → Grade
- ID, Name → Grade
- Etc,

Some Potential FDs may be "apparent", but don't assume presence if you are not the designer.

Language

- Let *X → Y* be a functional dependency present on relation R

- Verbally, we may say any of the following…
  - X → Y *holds* on R
  - X *functionally determines* Y
  - Y is *functionally dependent* on X

- Additional:
  - X is *determinant*
  - Y is *dependent*

Check Understanding (1)

- Whenever any two tuples t_1 and t_2 of r agree on one attribute α, they also agree on another attribute β.
  - $t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$
  - This relation is denoted $\alpha \rightarrow \beta$

- Check understanding

  F: $X \rightarrow Y$

X	Y
a	b
a	?

Check Understanding (2)

- Whenever any two tuples t_1 and t_2 of r agree on one attribute α, they also agree on another attribute β.
  - $t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$
  - This relation is denoted $\alpha \rightarrow \beta$

- Check understanding:

  F: $X \rightarrow Y$

X	Y
a	b
?	b

Check Understanding (3)

- Whenever any two tuples t_1 and t_2 of r agree on one attribute α, they also agree on another attribute β.
  - $t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$
  - This relation is denoted $\alpha \rightarrow \beta$

- Check understanding

  F: $X, Y \rightarrow X$

  F: $X \rightarrow X$

- Note: Functional dependencies like these are trivial

Generalization

- Before we said whenever any two tuples t_1 and t_2 of r agree on **one** attribute α, they also agree on another attribute β.
- More accurately, a functional dependency can be defined between any two **sets** of attributes. I.e., value(s) of a set of attributes determines value(s) of another set of attributes.

- Examples:
  - $XY \rightarrow WZ$
  - $XW \rightarrow Z$
  - $Z \rightarrow XQ$

Recall from before slides

- ID, Code → Grade
- ID, Name → Grade
- {ID} →{Phone , Major}

- **Constraint** between two sets of attributes; might be enforced across the entire database schema (even after decomposition)

Where are the FDs in DBMSs?

- **Database Design Phase:** Designers explicitly list functional dependencies on paper or in documentation during the initial design phase to help normalize the database .
  - Once implemented, functional dependencies are typically not explicitly stated in a live database system, they exist/ are enforced through **schema** constraints.

STUDENTS				
ID	**Phone**	**Major**	**Prof**	**Grade**
1	237-4539	Comp Sci	Smith	A
2	427-7390	Chemistry	Turner	B
3	237-4539	Comp Sci	Clark	B
4	388-5183	Physics	James	A
5	371-6259	Decision Sci	Cook	C
6	823-7293	Mathematics	Lamb	B
7	823-7293	Mathematics	Bond	UN
8	237- 4539	Comp Sci	Cross	UN
9	839-0827	English	Broes	C

Instance r0
Legal?

In Lecture Demo: Students

- Suppose we need to define the FDs of this first draft of a relation
  - What can we know about the columns?
  - Could each ID have a unique phone number and major?

STUDENTS					
ID	**Course**	**Phone**	**Major**	**Prof**	**Grade**

M: Note this is already a bit backwards…

In Lecture Demo: Students

- Every ID has a unique phone number and major?
  - We can say {ID} →{Phone , Major}

- Other relations between columns:
  - Every course has a unique professor {Course} → {Prof}
  - Every ID and course has a unique grade {ID , Course} → {Grade}

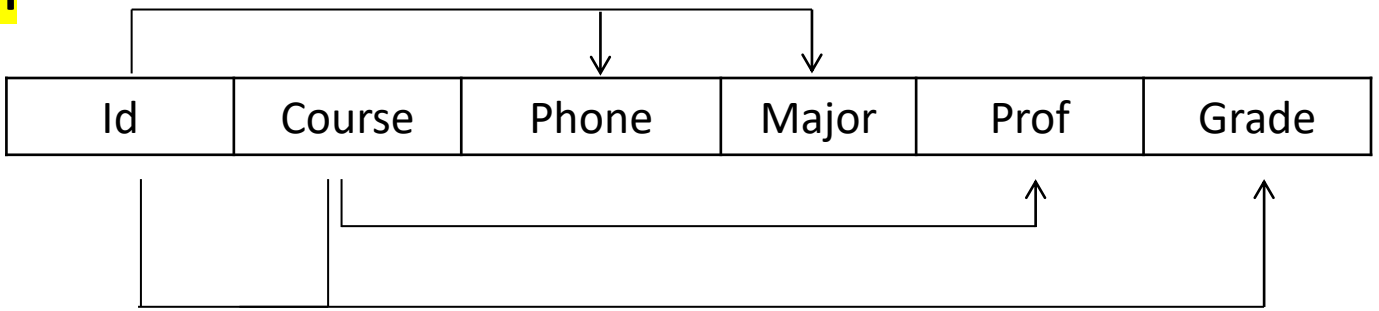
STUDENTS					
ID	**Course**	**Phone**	**Major**	**Prof**	**Grade**

We may denote the attributes sets with/without curly brackets.
Generally, with curly brackets, attributes are comma separated {X,Y} = XY
The order of the attribute sets doesn't matter. ZY = YZ OR {Z,Y} = {Y, Z}

In Lecture Demo: Students

- **Functional Dependencies**
  - {ID} →{Phone , Major}
  - {Course} → {Prof}
  - {ID , Course} → {Grade}

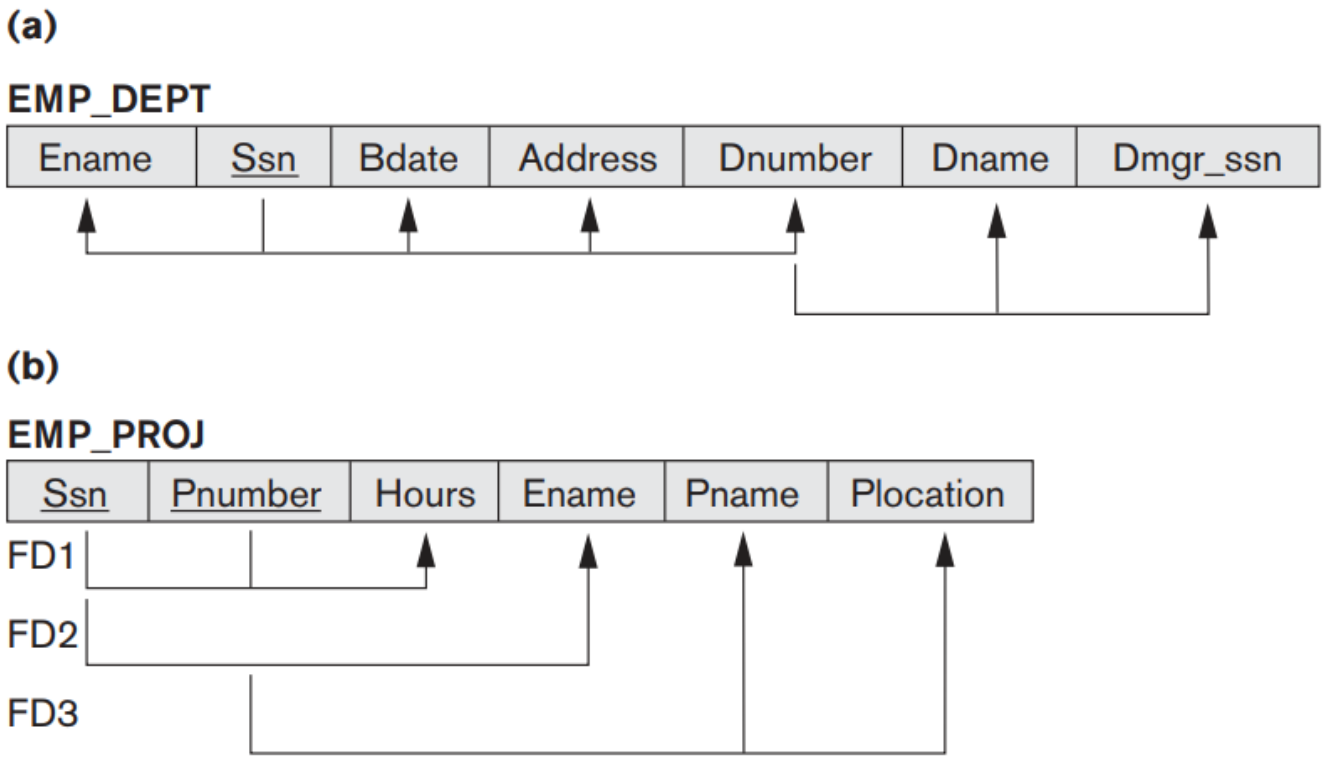
- **Diagram**



- Each *horizontal line* represents a FD
  - **Each** Left-hand side attr. links to the horizontal line via vert. lines
  - **Each** Right-hand side attr. links to the horizontal line by vert. lines
    - ending with arrows (pointing toward said rhs attributes)

Dependency Diagram*

- More Examples of dependency diagrams



Usecase One: basis for Decompositions

FUNCTIONAL DEPENDENCIES

Recall the Goal of Database Design

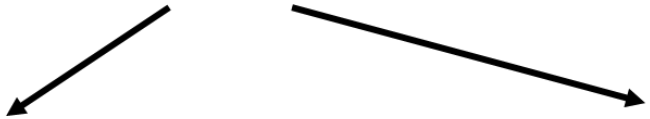
- Continuing from the schema we built from E/R modelling, we are going to expand more on:
  - Decide when a relation R has "good" form (and devise a design theory)

- Case
  - If already "good", great
  - If a relational schema is "not good"… improve by **decomposition** into set of relations {R1, R2, ..., Rn} such that
    - Each relation is in good form
    - Decomposition is a lossless-join decomposition
    - *Idea: many problems arising from redundancy can be addressed by replacing a relation with a collection of 'smaller' relation.*

Decomp. Candidate? (Wages → Hours)!

- A decomposition of a relation schema R consists of replacing the relation schema by two (or more) relation schemas that each contains a subset of attributes of R and together include all attributes in R.

Id	name	age	rating	Hourly_wages	Hours_worked
123-22-3666	Peter	48	8	10	40
231-31-5368	Paul	22	8	10	30
131-24-3650	Mary	35	5	7	30
434-26-3751	David	35	5	7	32
612-67-4134	Ada	35	8	10	40

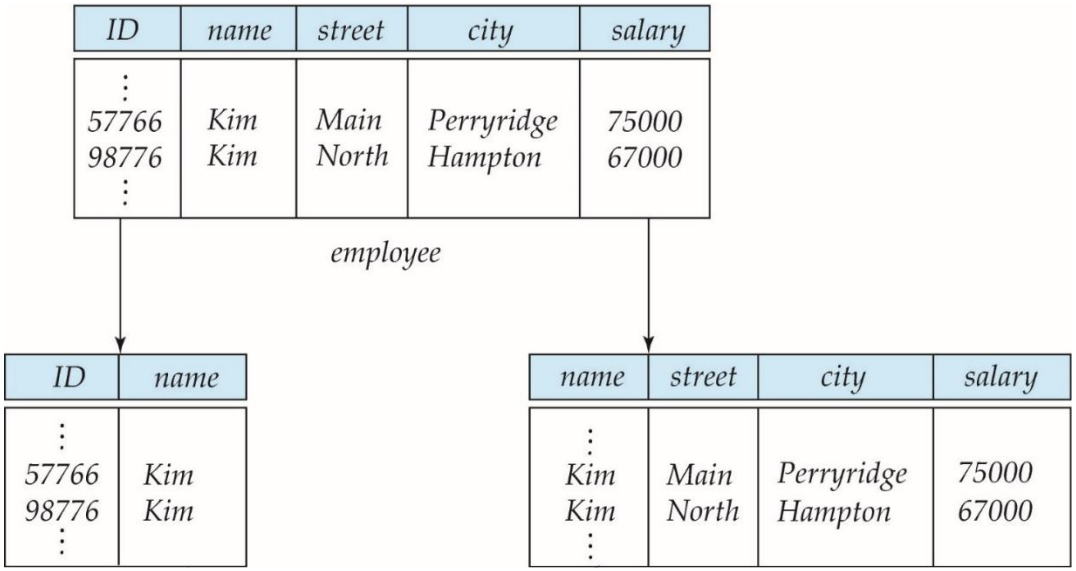


Id	name	age	rating	Hours_worked
123-22-3666	Peter	48	8	40
231-31-5368	Paul	22	8	30
131-24-3650	Mary	35	5	30
434-26-3751	David	35	5	32
612-67-4134	Ada	35	8	40

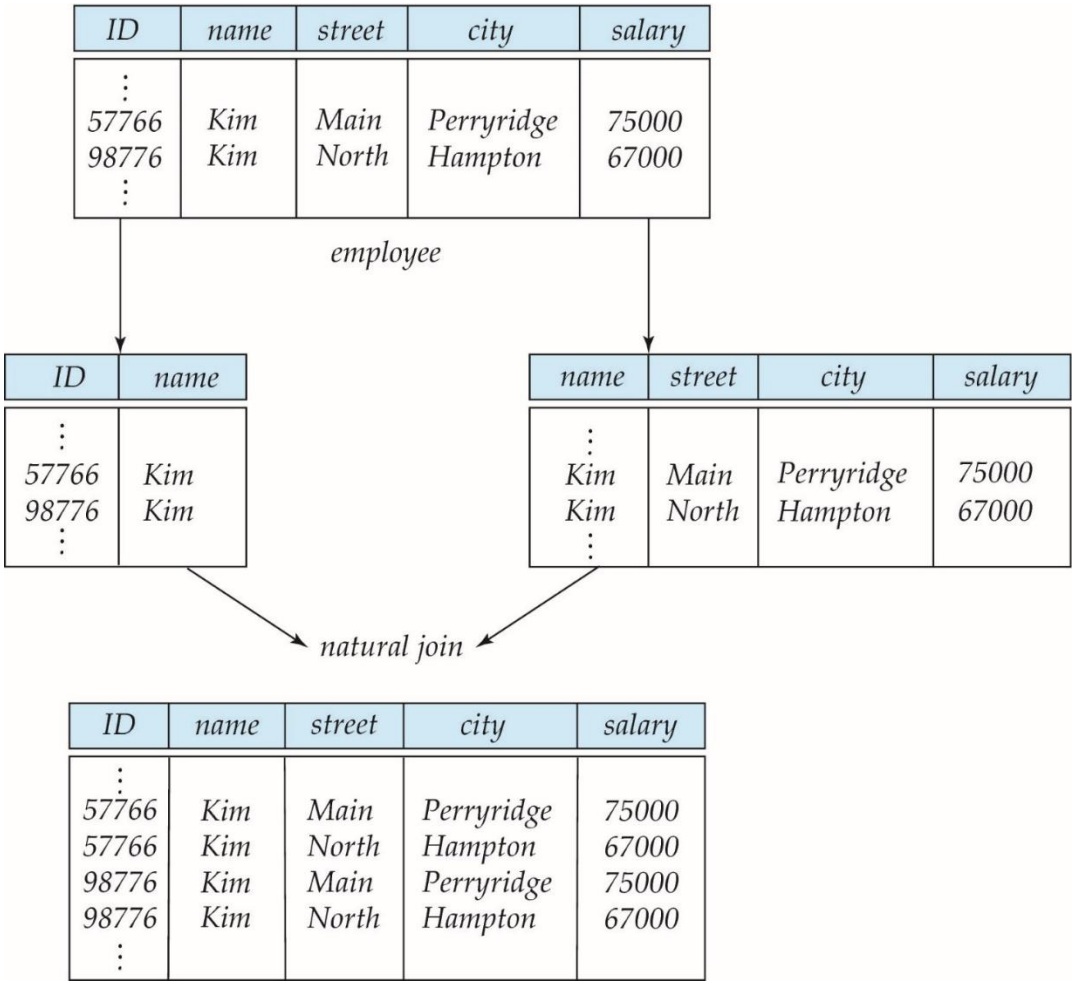
rating	Hourly_wages
8	10
5	7

Functional dependency:
- rating determines Hourly_wages

Demo (1) - Decomposition



Demo (2) – Original "info" Recoverable?



Decomposition → Lossless-Join?

- Simple example of Lossy-Join (Non-Lossless Join):
  - Decomposition of *R = (A, B) into* $R_1 = (A)$ *and* $R_2 = (B)$

A	B
α	1
α	2
β	1

r

A
α
β

$\prod_A(r)$

B
1
2

$\prod_{B(r)}$

A	B
α	1
α	2
β	1
β	2

$\prod_A (r) \bowtie \prod_B (r)$

- Lossless-Join decomposition:
  - For all possible relation instance *r* on schema *R*

$$r = \prod_{R1}(r) \bowtie \prod_{R2}(r)$$

Given $A \rightarrow B$ and $B \rightarrow C$...

what do we know about $A \rightarrow C$?

ON ALL OTHER INFERABLE FDS

Inferring Other FDs

- M: Usually, the schema designer will only specify the functional dependencies that are **semantically obvious**.

- It's true that given a set F of functional dependencies, there are other functional dependencies that are **logically implied** by F, and are naturally expected to be all enforced.

- F |= X→ Y
  - Denotes that set of FDs F infers X → Y if all relation instances satisfying F also satisfies X → Y .

- Example:
  - F = $A \rightarrow B$, $B \rightarrow C$
  - F |= $A \rightarrow C$

Inference Rules

- There are three basic inference rules for functional dependencies

- **Armstrong's Axioms**
  - **It's** primary rules/axioms as follows
  - *Note: α, β, γ are all (nonempty) sets of attributes*

- Rule 1 (**reflexivity**)
  - if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$
- Rule 2 (**augmentation**)
  - if $\alpha \rightarrow \beta$, then $\gamma\ \alpha \rightarrow\ \gamma\ \beta$
- Rule 3 (**transitivity**)
  - if $\alpha \rightarrow \beta$, and $\beta \rightarrow \gamma$, then $\alpha \rightarrow\ \gamma$

Practice

- $R = (A, B, C, G, H, I)$
  - $F = \{ A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H \}$
  - Can these following 3 FDs can be inferred/ deduced?
  - Are there more?

$A \rightarrow H$

$AG \rightarrow I$

$CG \rightarrow HI$

Rule 1 (**reflexivity**)

if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$

Rule 2 (**augmentation**)

if $\alpha \rightarrow \beta$, then $\gamma\ \alpha \rightarrow \gamma\ \beta$

Rule 3 (**transitivity**)

if $\alpha \rightarrow \beta$, and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$

Language: A closure of F

- Note: We denote by *F* the set of functional dependencies that are specified on relation schema *R*.
  - FDs logically imply other FDs.
  - Set of **all** FDs logically implied by F is the **closure** of F.
    - F+ denotes a **closure** of F
    - F+ is a superset of F

- In general, given F, we can find F+ (the closure of F) how?
  - by repeatedly applying Armstrong's Axioms.

- Armstrong's Axioms are proven to be **sound** and **complete**
  1. Sound = generates only functional dependencies that hold
  2. Complete = generates all functional dependencies that hold

Armstrong's Axioms (Cont.)

- Additional Rules we inferred from Armstrong's axioms.
  - Rule 4 (**additivity**):
    - If $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\ \gamma$ holds
  - Rule 5 (**projectivity**):
    - If $\alpha \rightarrow \beta\ \gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds
  - Rule 6 (**pseudo-transitivity**):
    - If $\alpha \rightarrow \beta$ holds and $\gamma\ \beta \rightarrow \delta$ holds, then $\alpha\ \gamma \rightarrow \delta$ holds

- **Note: common variation on names**
  - Additivity aka Union, Projectivity aka Decomposition